

Introdução à Programação

Aula 06 – Estruturas de dados básicas

Prof. Jacson Rodrigues Barbosa

jacson_rodrigues@ufg.br

Estruturas de dados compostas em GO

- As estruturas de dados compostas são os tipos de dados formados à partir dos tipos primitivos:
 - **Strings:** uma string é uma cadeia imutável de caracteres (bytes) representados usando a codificação UTF-8.
 - **Arrays:** uma coleção com tamanho fixo de variáveis do mesmo tipo e que são acessadas através de índices.
 - **Slices:** é uma abstração de um array, uma fatia, ou seja, uma view para um array. Um slice é indexado e possui um tamanho, mas sua vantagem é que podemos mudar esse tamanho com facilidade.
 - **Maps:** equivalem aos arrays associativos, onde temos uma coleção de pares “chave → valor”.
 - **Structs:** é uma estrutura de dados heterogênea. Abstratamente é um registro composto de campos de diferentes tipos de dados, mas logicamente relacionados.

Arrays

- Array unidimensional (vetor) é uma estrutura homogênea de dados que armazena uma sequência de variáveis (elementos), todas do mesmo tipo;
- Acessados individualmente através de um número sequencial que denominamos índice do array;
- A quantidade de elementos (tamanho do array) e seu tipo devem ser definidas no momento da declaração e são imutáveis;
- Cada elemento do vetor, por meio do seu índice, pode ser acessado como uma variável individual.



Operações com Array

- Declaração de Arrays
 - `var nota[3] float64`
 - `var nota = [3] float64{5,6.5,7}`
- Atribuição de valores para os elementos

`nota[0] = 5`

`nota[1] = 6.5`

`nota[2] = 7`

- Acessando os elementos

`//imprimir um elemento`

`fmt.Println(nota[1])`

`//imprimir todo o vetor`

`fmt.Println(nota)`

Operador Range e o Identificador Blank

- Iteração sobre os elementos de um *array*:

```
for i:= 0, i < len(nota); i++ {  
    fmt.Printf("índice = %d e Valor = %f",i,nota[i])  
}
```

- Operador *range* devolve, a cada iteração do *for*, o índice e o valor do array:

```
for i,v := range nota {  
    fmt.Printf("Índice = %d e Valor = %f",i,v)  
}
```

```
package main
import "fmt"
func main() {
    const numNotas int = 3 // Quantidade de notas
    var (
        nota [numNotas]float64 // Array de notas
        soma float64 = 0
    )
    // Parte 1 – Leitura
    for i := 0; i < numNotas; i++ {
        fmt.Printf("Informe a nota %d: ", i)
        fmt.Scan(&nota[i]) // Armazena nota no array
        soma += nota[i] // Calcula a soma das notas
    }

    for i, v := range nota {
        fmt.Printf("Nota %d = %f\n", i, v)
    }
    fmt.Printf("Soma da notas %f: ", soma)
}
```

Operador Range e o Identificador Blank

- Caso alguns desses valores de retorno não sejam necessários, podemos ignorá-los através do identificador blank, o underscore. Por exemplo, caso o índice do elemento não seja necessário, podemos refazer a repetição anterior:

```
for _,v := range nota {  
    fmt.Printf("Valor = %f",v)  
}
```

Exercício 1

Leia 3 notas de um aluno, armazene em um array e calcule a média Geral. Logo após, relacione as notas que estão acima da média.

Exercício 2

Faça um programa que leia 5 valores inteiros, armazeno-os em um vetor, calcule e apresente a soma destes valores.

Exercício 3

Faça um programa que leia 10 valores reais e os apresente na ordem inversa entrada.

Saiba mais

- Links
 - [Tutorialspoint](#)
 - [Geeks for Geeks](#)
 - [Go by Example](#)
 - [A tour of GO](#)

Referências

[1] Introdução à Linguagem de Programação Go,
Prof José Cintra, 2020